מה נשאר לבעלים: רשימה מסודרת, שלב אחרי שלב

עודכן: 11 בנובמבר 2026, בסוף סבב ההשלמות על הענף `claude/dreamy-knuth-hwmgsk`.

כל מה שאפשר לעשות בקוד כבר נעשה, נבדק ונדחף. מה שכתוב כאן דורש אתכם: גישה ל-GitHub, לשרת, לחשבונות אצל ספקים, או החלטה עסקית או משפטית. עובדים לפי הסדר: שלבים 1–4 חוסמים (בלעדיהם השינויים לא עולים לאוויר), השאר אפשר לעשות בהדרגה.

סימון: ☐ = לעשות. 🧑‍💻 = בממשק של viper fleet. 🖥️ = בשרת. 🐙 = ב-GitHub. 📄 = מסמך/החלטה.

שלב 1 🐙: למזג את העבודה ל-`main`

1. ☐ לפתוח Pull Request מהענף `claude/dreamy-knuth-hwmgsk` אל `main`.
2. ☐ לחכות שה-CI יעבור (lint, טיפוסים, knip, יחידה, build ותקציב גודל, e2e, בדיקות מסד, טיפוסי מסד).
3. ☐ למזג. דחיפה ל-`main` מעלה ל-staging בלבד (אם הוגדר, ראו שלב 2). ייצור עולה רק בשלב 3.

שלב 2 🐙: משתנים וסודות ב-GitHub (פעם אחת, חוסם פריסה)

GitHub: Settings ← Secrets and variables ← Actions-ב.

Variables (לשונית Variables):

שם	מה לשים	הערה
`DEPLOY_HOST`	`user@address` של השרת	עדיף משתמש פריסה ולא root
`VPS_HOST_KEY`	השורה שמחזיר `ssh-keyscan -t ed25519 <address>`	מוצמד, נגד התחזות לשרת
`PRODUCTION_SITE_URL`	כתובת האתר, למשל `https://app.example.co.il`	לבדיקת העשן אחרי פריסה
`PRODUCTION_FLEET_URL`	כתובת ה-Supabase של הייצור	חובה ל-build
`PRODUCTION_FLEET_ANON_KEY`	מפתח ה-anon של Supabase הייצור	מומלץ: יש ערך ברירת מחדל ב-workflow, עדיף משתנה
`PRODUCTION_DB_CONTAINER`	שם קונטיינר ה-Postgres של הייצור	רק אם יש יותר מ-stack אחד על השרת (staging)
`PRODUCTION_RUNTIME_CONTAINER`	שם קונטיינר ה-edge runtime של הייצור	כנ"ל

Secrets (לשונית Secrets):

שם	מה לשים
`VPS_SSH_KEY`	המפתח הפרטי (ed25519) שמורשה להתחבר ל-`DEPLOY_HOST`

☐ Settings ← Environments ← `production` ← Required reviewers: **Environment**: להוסיף את עצמכם, כדי שכל פריסה לייצור תחכה לאישור.

staging (לא חובה, מומלץ): לפי `deploy/STAGING.md`, עם המשתנים `STAGING_WEB_DIR`, `STAGING_SITE_URL`, `STAGING_FLEET_URL`, `STAGING_FLEET_ANON_KEY`, `STAGING_FLEET_DIR`, `STAGING_DB_CONTAINER`, `STAGING_RUNTIME_CONTAINER`. בלי `STAGING_WEB_DIR` הפריסה ל-staging פשוט מדולגת.

שלב 3: פריסה לייצור, כולל כל המיגרציות

דרך א' (מומלצת) 🐙: בלחיצה

1. ☐ Run workflow ← **Promote to production** ← Actions ← GitHub ← להקליד `production`.
2. ה-workflow עושה הכול לבד: CI מלא, build, העלאה לשרת, **גיבוי מלא של המסד**, הרצת כל המיגרציות שעוד לא רצו (ורישום שלהן ב-`vf_meta.ui_migrations`), פריסת ה-edge functions (`fleet-ai`, `public-api`, `billing-documents`), ובדיקת עשן.
3. ☐ לאשר את הסביבה `production` כשמתבקשים.
4. אם משהו נשבר: **Roll back** ← Actions (או `sh deploy/deploy.sh rollback`). הגיבוי שלפני המיגרציות נשמר בשרת ב-`/root/vf-backups/pre-migration-*.dump`.

אחרי דרך א' מדלגים ישר לשלב 4.

דרך ב' 🖥️: ידנית בשרת (אם אין עדיין משתני GitHub)

3.1 לבדוק מה כבר רץ. ב-Supabase Studio ← SQL Editor (או `psql`), להדביק ולהריץ. התוצאה היא רשימת המיגרציות שעוד **לא** רצו בייצור (אם הטבלה לא קיימת, שום מיגרציה לא נרשמה; עוברים ל-3.3):

```
        select m.name as not_applied_yet
        from (values
          ('20261104120000_super_admin.sql'),
          ('20261104130000_super_admin_permissions.sql'),
          ('20261106120000_fleet_stage1.sql'),
          ('20261106130000_fleet_stage2.sql'),
          ('20261106140000_fleet_stage3.sql'),
          ('20261106150000_fleet_stage4.sql'),
          ('20261106160000_fleet_stage5.sql'),
          ('20261109120000_security_hardening.sql'),
          ('20261109130000_dispatch_tracking_links.sql'),
          ('20261110120000_gov_registry_sync.sql'),
          ('20261110130000_messaging.sql'),
          ('20261110140000_violation_transfer.sql'),
          ('20261110150000_privacy.sql'),
          ('20261110160000_safety_officer.sql'),
          ('20261110170000_driver_area.sql'),
          ('20261111100000_public_api.sql'),
          ('20261111110000_public_api_admin.sql'),
          ('20261112100000_billing_documents.sql'),
          ('20261113100000_client_metrics.sql'),
          ('20261114100000_role_guard.sql'),
          ('20261114110000_billing_integrity.sql'),
          ('20261114120000_privacy_rights.sql'),
          ('20261114130000_fleet_save.sql'),
          ('20261114140000_vehicle_archive.sql'),
          ('20261114150000_capabilities.sql'),
          ('20261114160000_inspection_default.sql'),
          ('20261115100000_fleet_ai.sql'),
          ('20261115110000_safety_inbox.sql'),
          ('20261115120000_driver_checkin.sql'),
          ('20261115130000_fuel_checks.sql'),
          ('20261115140000_driver_write_limits.sql'),
          ('20261115150000_file_limits.sql'),
          ('20261115160000_queue_expiry.sql'),
          ('20261115170000_impersonation.sql'),
          ('20261115180000_webhook_hardening.sql'),
```

```
          ('20261115190000_alert_cases.sql'),
          ('20261115200000_rls_performance.sql'),
          ('20261115210000_api_keys_server.sql'),
          ('20261115220000_org_plans.sql'),
          ('20261115230000_pool_vehicles.sql'),
          ('20261115240000_automations.sql'),
          ('20261115250000_recipient_test.sql'),
          ('20261115260000_new_drivers.sql'),
          ('20261115270000_dispatch_contact.sql'),
          ('20261116100000_dashboards.sql'),
          ('20261116110000_fuel_review.sql'),
          ('20261116120000_work_order_close.sql'),
          ('20261116130000_dispatch_failures.sql'),
          ('20261116140000_dispatch_route_order.sql'),
          ('20261116150000_automation_rest_day.sql'),
          ('20261116160000_vehicle_handovers.sql'),
          ('20261116170000_sim_cards.sql'),
          ('20261116180000_fuel_card_limits.sql'),
          ('20261116190000_branded_email.sql'),
          ('20261116200000_tires.sql'),
          ('20261116210000_vendor_prices.sql'),
          ('20261116220000_fleet_benchmarks.sql'),
          ('20261116230000_asset_tags.sql'),
          ('20261116240000_training_courses.sql'),
          ('20261116250000_market_prices.sql'),
          ('20261116260000_driver_fueling.sql'),
          ('20261116270000_checkin_source.sql'),
          ('20261117100000_missed_check_alerts.sql'),
          ('20261117110000_driver_disputes.sql'),
          ('20261117120000_toll_charges.sql'),
          ('20261117130000_orphan_files.sql')
        ) as m(name)
        where not exists (select 1 from vf_meta.ui_migrations u where u.name = m.name)
        order by m.name;
```

3.2 גיבוי לפני הכול (בשרת, ב-SSH; להחליף `<db>` בשם קונטיינר ה-Postgres, שמופיע ב-`docker ps` עם התמונה `supabase/postgres`):

```
        mkdir -p /root/vf-backups && chmod 700 /root/vf-backups
        docker exec <db> pg_dump -U postgres -d postgres -Fc > /root/vf-backups/manual-$(date -u
+%Y%m%dT%H%M%SZ).dump
        ls -lh /root/vf-backups | tail -3
```

3.3 להריץ את כל המיגרציות החסרות בפקודה אחת. הקובץ `docs/owner/apply-migrations.sql` (במאגר) מכיל את כל 66 המיגרציות לפי הסדר. הוא בודק לפני כל אחת אם היא כבר רצה ומדלג עליה, מריץ את החסרות, ורושם אותן. הרצה כפולה בטוחה (נבדק: כל המיגרציות רצו פעמיים על מסד בדיקה בלי שגיאה). ממחשב שיש בו את המאגר:

```
        scp docs/owner/apply-migrations.sql <user@server>:/tmp/apply-migrations.sql
        ssh <user@server>
        docker exec -i <db> psql -X -v ON_ERROR_STOP=1 -U postgres -d postgres < /tmp/apply-
migrations.sql
```

בפלט מופיעה שורה לכל מיגרציה: `Already applied: ...` או `Applying: ...`, ובסוף `Migrations done.`

> את הקובץ הזה צריך `psql` (הוא משתמש ב-`\if`). ה-SQL Editor של Studio לא יריץ אותו. אם חייבים Studio: פותחים כל קובץ חסר מ-`supabase/migrations/` לפי הסדר של 3.1, מדביקים ומריצים אותו, ומיד אחריו: `insert into vf_meta.ui_migrations (name) values ('<שם הקובץ>') on conflict do nothing;` לפני כן, פעם אחת:

```
create schema if not exists vf_meta; create table if not exists vf_meta.ui_migrations (name
text primary key, applied_at timestamptz not null default now());
```

3.4 לרענן את ה-API ולפרוס את הפונקציות:

```
docker exec -i <db> psql -U postgres -d postgres -c "notify pgrst, 'reload schema';"
DEPLOY_HOST=<user@server> sh deploy/functions.sh      # ממחשב עם המאגר
```

3.5 להעלות את האתר: `DEPLOY_HOST=<user@server> FLEET_URL=<כתובת ה-Supabase> sh deploy/deploy.sh`

3.6 לוודא שהכול רץ (בכל אחת מהדרכים)

```
-- צריך להחזיר 66, ואת ארבע האחרונות של הסבב הזה:
select count(*) from vf_meta.ui_migrations;
select name, applied_at from vf_meta.ui_migrations order by name desc limit 4;
-- 20261117130000_orphan_files.sql, 20261117120000_toll_charges.sql,
-- 20261117110000_driver_disputes.sql, 20261117100000_missed_check_alerts.sql

-- המשימות המתוזמנות (pg_cron) שהמיגרציות יוצרות:
select jobname, schedule from cron.job order by jobname;
-- בין היתר: vf_messaging (כל דקה) ו-vf_missed_checks (כל 10 דקות, חדש בסבב הזה)
```

מה עושות המיגרציות של הסבב הזה:

קובץ	מה הוא מוסיף
`20261117100000_missed_check_alerts.sql`	הודעה למנהל כשרכב יצא בלי בדיקת בוקר (ADM-31), כל 10 דקות
`20261117110000_driver_disputes.sql`	הנהג רואה את הציון שלו ומערער על אירוע; המנהל מכריע בתיבת הבטיחות (SAF-05, SEC-23)
`20261117120000_toll_charges.sql`	שמירת דוחות כביש 6/חניה, סימון שימוש פרטי ומרכזי עלות (ISR-06)
`20261117130000_orphan_files.sql`	איתור קבצים שאף רשומה לא מצביעה עליהם (SEC-17)

ואם הן עוד לא רצו, גם 27 המיגרציות של הסבבים הקודמים (מ-`20261115190000_alert_cases.sql` עד `20261116270000_checkin_source.sql`): ניהול אירועי התראה, RLS מהיר, מפתחות API בשרת, חבילות, רכבי מאגר (יוצרת את ההרחבה `btree_gist` לבד), אוטומציות, דשבורדים במסד, מסירת רכב, SIM, צמיגים, מחירון ספקים, השוואה לציים, תגי BLE, הדרכות, מחירי שוק, תדלוק מהפורטל וצ'ק-אין.

שלב 4: פקודות SQL חד-פעמיות אחרי המיגרציות (להעתקה)

ב-SQL Editor ← Studio (רץ כ-postgres):

4.1 ☐ "פלפל" למפתחות ה-API (חובה אם משתמשים ב-API הציבורי או ב-MCP). קודם מייצרים מחרוזת אקראית במחשב: `openssl rand -base64 48`, ואז:

```
select public.api_key_set_pepper('<המחרוזת שיצאה, לפחות 32 תווים>');
```

לשמור את המחרוזת במקום בטוח. אם מחליפים אותה, כל מפתחות ה-API הקיימים מפסיקים לעבוד.

4.2 ☐ חבילה לכל לקוח (BIZ-02). כל ארגון קיים מתחיל ב-`pro`. כדי לשנות (`tracking`, `fleet` או `pro`):

```
select id, name, plan from public.organizations order by name;
update public.organizations set plan = 'fleet' where id = '<org id>';
```

(מתוך הממשק זה נעשה בניהול השירות, למנהלי פלטפורמה.)

4.3 ☐ מפתח Anthropic ל-AI (`docs/AI.md`) 👩‍💻: מסך הגדרות ← AI, מחובר כמנהל פלטפורמה. (מ-Studio הפונקציה `ai_set_secret` מסרבת, כי היא בודקת שאתם מנהלי פלטפורמה מחוברים.)

4.4 ☐ בדיקה שהכול נקלט:

```
select name from vault.secrets order by name;   -- api_key_pepper, גם המפתח של ה-AI ואחרי 4.3
```

שלב 5 👩‍💻: הגדרות במסכים (בלי קוד), אחרי הפריסה

מהסבב הזה:

1. ☐ **נמעני התראות** (צי ← נמעני התראות, `/fleet/alerts`): לסמן אצל המנהל את הסוג החדש **"בדיקת בוקר שלא בוצעה"**. ההודעה יוצאת רק בארגון שיש לו צ'ק-ליסט פעיל בתדירות "יומית", על רכב שנהג לקח היום (צ'ק-אין או שיוך שהתחיל היום) ולא נעשתה בו בדיקה תוך 30 דקות. נשלחת רק כשערוץ שליחה מחובר (שלב 8).
2. ☐ **הגדרות בטיחות** (צי ← בטיחות נהגים ← "הגדרות בטיחות"): לבדוק עם קצין הבטיחות את הכללים לרכב כבד ולאוטובוס, ולהוסיף היתרים זמניים (למשל היתר החירום) עם תאריכים. סוג הרכב נקבע לפי ה"קטגוריה" של המכשיר ב-Traccar (truck = כבד, bus = אוטובוס): ☐ לוודא שהקטגוריה של כל משאית ואוטובוס נכונה.
3. ☐ **מרכזי עלות לרכבים** (בשביל דוח האגרות): בעריכת הרכב, שדה "מרכז עלות" (עובר למסד הצי בשמירה). לעדכון של הרבה רכבים בבת אחת אפשר גם SQL:

```
update public.vehicles set cost_center = 'מכירות' where plate_number in ('12-345-67', '23-456-78');
```

4. ☐ **דוח האגרות** (דוחות ← כבישי אגרה וחניה): לטעון את הדוח החודשי של כביש 6 / מנהרות / חניה, לסמן שימוש פרטי, וללחוץ "שמירת הדוח". דוח שנטען פעמיים לא נשמר פעמיים.
5. ☐ **מחירון ספקים** (צי ← ספקים, `/fleet/vendors` ← אייקון "מחירון מוסכם" בשורת הספק): אפשר עכשיו לייבא את המחירון מקובץ CSV או Excel.
6. ☐ **קבצים יתומים** (צי ← פרטיות עובדים, `/fleet/privacy`; רק בעלים או אדמין): "חיפוש קבצים כאלה", פעם ברבעון, ומחיקה אחרי עיון.
7. ☐ **פורטל הנהג**: כדי שנהג יראה את "הציון שלי", המשתמש שלו צריך להיות מקושר לכרטיס הנהג, והנהג צריך שיוך לרכב (או צ'ק-אין). כדאי להודיע לנהגים שהם יכולים לערער על אירוע, ושהערעורים מגיעים לתיבת הבטיחות עם מסנן "ערעורים".
8. ☐ **לוח השנה של הטיפולים** (צי ← תחזוקה, `/settings/maintenances` ← "לוח שנה"): אין מה להגדיר. טיפול לפי ק"מ מוצב לפי קצב הנסיעה של 30 הימים האחרונים.
9. ☐ **מפת החום במפה החיה**: כפתור הלהבה ליד כפתור ההרצה אחורה. אין מה להגדיר.

מהסבבים הקודמים (אם עוד לא נעשה):

10. ☐ יעד תגובה לאירועי התראה: fleet/alerts/response/.
11. ☐ איש קשר לנהגים (שם וטלפון): fleet/users/.
12. ☐ לסמן רכבי מאגר.

13. ☐ ברירת המחדל של הדשבורד: לפתוח את הדשבורד ← "פעולות נוספות" ← לפרסם מחדש לארגון. אחרי שכל הארגונים עברו, למחוק את `web.dashboardDefault` מהגדרות השרת של Traccar.
14. ☐ רכבי ליסינג: בדף העריכה של רכב "ליסינג" למלא תחילת חוזה, ק"מ בתחילתו, מכסת ק"מ ומחיר לק"מ חריגה.
15. ☐ שרשרת קירור: טווח טמפרטורה לכל רכב בעמוד החיישנים.
16. ☐ בדיקת בוקר: צ'ק-ליסט פעיל בתדירות "יומית".
17. ☐ תוכניות טיפול: ליצור מהתבניות ולשייך לרכבים.
18. ☐ צ'ק-אין: להדפיס את קוד ה-QR של כל רכב (דף הרכב) ולהדביק בתא הנהג.
19. ☐ בדיקות דלק: מחיר הליטר המפוקח בכל חודש.

שלב 6 🖥️: השרת (פעם אחת)

1. ☐ גיבוי לילי: `deploy/backup.sh` ב-cron על השרת, עם העתקה מחוץ לשרת. עם staging על אותו שרת, להוסיף `DB_CONTAINER=...` לשורת ה-cron.
2. ☐ כותרות אבטחה (SEC-10): `sh deploy/security-headers.sh traefik` (או `nginx` / `caddy`), ולהדביק בהגדרות ה-reverse proxy לפי `deploy/SECURITY-HEADERS.md`.
3. ☐ הקשחה (SEC-11, SEC-19): פורט 8082 של Traccar רק על 127.0.0.1, חומת אש לפורטי המכשירים, Studio רק דרך מנהרת SSH ובלי סיסמת ברירת המחדל. להריץ ממחשב רגיל `sh deploy/check-exposure.sh <ip> <url>` ולמלא את הטבלה ב-`deploy/SERVER-HARDENING.md`.
4. ☐ staging לפי `deploy/STAGING.md` (לא חובה).

שלב 7 🖥️: הגדרות בשרת Traccar

1. ☐ `database.historyDays` לפי מדיניות השמירה, וסינון דוחות מתוזמנים (SEC-02, SEC-12).
2. ☐ מגבלות מהירות לפי כביש (SAF-06, `docs/SPEED-LIMITS.md`). בצי אמיתי עדיף שרת Overpass משלכם.
3. ☐ `web.geocoderUrl` : Nominatim על השרת שלכם או GovMap (ISR-10). היום משתמשים ב-Nominatim הציבורי.
4. ☐ פרטי רכז נגישות: `accessibilityContact` , `accessibilityPhone` , `accessibilityEmail` , ובאתר `website/src/config.js` ← `site.accessibility` (A11-07).
5. ☐ Mobileye: לכל התקנה לחבר את היציאות או להגדיר attribute מחושב (`docs/MOBILEYE.md`).

שלב 8: מפתחות וחשבונות אצל ספקים

1. ☐ **ערוצי שליחה** 👩‍💻 (ניהול השירות ← הודעות): מפתחות WhatsApp (Meta), SMS (Vonage) ומייל (Resend). בלי זה שום התראה לא יוצאת, גם לא החדשה של בדיקת הבוקר. בערוץ המייל למלא "כתובת האתר", להגיש את תבנית הוואטסאפ לאישור ב-Meta ולאמת את הדומיין ב-Resend (`docs/MESSAGING-TEMPLATES.md`).
2. ☐ **Morning** (חשבוניות): מפתח API, חיבור לרשות המסים, Cardcom, ו-`BILLING_DOCUMENTS_URL` לתשלום באשראי. לבדוק את `/account/token` לפני שימוש אמיתי.
3. ☐ **AI**: מפתח Anthropic (שלב 4.3), ושורה על ספק ה-AI במרשם העיבוד.
4. ☐ **תצפיתיות** (`docs/OBSERVABILITY.md`): להחליט GlitchTip או Sentry, ואיזו אנליטיקה; אחר כך משתני `*_ERROR_DSN` , `*_ANALYTICS_*` , `ERROR_SOURCEMAPS_*` והסוד `ERROR_AUTH_TOKEN` , ולהוסיף ל-CSP.

5. ☐ **מצלמות** (SAF-03): לבחור ספק לפי `docs/DASHCAMS.md`.
6. ☐ **מזג אוויר** (SAF-07): Open-Meteo חינמי רק לשימוש לא מסחרי; לשימוש מסחרי מנוי או השירות המטאורולוגי.
7. ☐ **כניסת נהג בטלפון וקוד** (ADM-31): דורש ספק SMS ל-Supabase Auth (Twilio, Vonage ועוד). כשיש ספק, זו הגדרה ב-Supabase ושינוי קטן במסך הכניסה.

שלב 9 📄: פרטיות, משפט ונגישות

1. ☐ עורך דין: ההודעה לעובדים, ההסכמה, מרשם העיבוד, ועכשיו גם הציון שהנהג רואה והערעור (SEC-23).
2. ☐ הסכמי עיבוד (DPA) עם Hostinger, ספק המפות והכתובות, Anthropic (אם AI פעיל), שירות הניטור ו-Morning.
3. ☐ לברר איפה מרכז הנתונים של השרת (`docs/PRIVACY-VENDORS.md`).
4. ☐ יועץ נגישות: בדיקת הנוסח (A11-07).
5. ☐ נהג חדש (ISR-12): לאמת את התקופות מול התקנות ולהחליט על מגבלת נוסעים (`src/common/fleet/newDriver.js`).
6. ☐ שעות נהיגה (ISR-07): קצין בטיחות מאשר את המספרים לרכב כבד ולאוטובוס, ואת נוסח ההיתרים.
7. ☐ השוואה לציים דומים (AI-09): סעיף הסכמה בחוזה לפני שמפעילים אצל לקוחות.
8. ☐ תרגום מסכי הנהג לערבית ולרוסית: שדובר שפת אם יעבור עליהם.

שלב 10 📄: החלטות עסקיות

1. ☐ חבילות ומחירים (BIZ-02): מה בכל חבילה, וחבילה לכל לקוח (שלב 4.2). כרגע החבילה מנחה את הממשק ולא נאכפת במסד.
2. ☐ BIZ-03, BIZ-04, BIZ-07 ב-`docs/business/DECISIONS.md`; פיילוט מעבר (`docs/business/MIGRATION-GUIDE.md`); השלמת `docs/business/SECURITY-OVERVIEW.md`.
3. ☐ מרכז אינטגרציות (INT-02): לבחור מחברים ראשונים ולהשיג מכל ספק API או פורמט קובץ.
4. ☐ אפליקציה בחנויות (INT-03): חשבונות מפתח ב-Apple וב-Google, ומחשב עם Xcode ו-Android Studio.
5. ☐ פורטל לקוח קצה (OPS-11): להחליט אם נותנים חשבונות ללקוחות של הלקוחות.
6. ☐ אוטומציות בלי אפליקציה פתוחה (AI-05) ו-API כתיבה (INT-01): צריך חשבון שירות ב-Traccar ו-worker בשרת.

שלב 11: שוטף

מתי	מה
כל חודש	מחיר הליטר המפוקח במסך בדיקות הדלק; טעינת דוח האגרות ושמירתו
כל שבועיים	`sh tools/upstream-review.sh` ורישום ב-`docs/UPSTREAM.md`
כל רבעון	תרגול שחזור מגיבוי; חיפוש קבצים יתומים (צי ← פרטיות עובדים)
כל ינואר	ערכי שווי השימוש של השנה מול רו"ח (ISR-05)
לפני כל פריסה לייצור	Actions ← Promote to production (גיבוי ומיגרציות נעשים לבד)

מה עוד פתוח בקוד, ולמה זה לא נעשה עכשיו

פריט	מה חסר	חוסם
ADM-31	כניסת נהג בטלפון וקוד	ספק SMS (שלב 8.7)
SAF-02, AI-05, INT-01, SEC-06	ריצה בשרת בלי אפליקציה פתוחה	חשבון שירות ב-Traccar + worker
SEC-13, SEC-16, SEC-22	claim ב-traccar-auth, הצפנת מזהים, שער JWT	עבודה בצד השרת (traccar-auth)
AI-02	תקציר יומי בוואטסאפ	שולח ותבנית מאושרת ב-Meta
SAF-03, SAF-04	קליפ וידאו לאירוע	בחירת ספק מצלמות
OPS-03, OPS-09	חלוקה בין רכבים, OCPP	שרת ניתוב / עמדות טעינה
OPS-05, ISR-04, ISR-06, ISR-07	ייבוא לחילן/מל"מ, פענוח טכוגרף, פורמטים מדויקים של מפעילים	מפרטים ודוגמאות קבצים אמיתיות
ISR-10	GovMap או geocoder משלכם	החלטה ורישום (שלב 7.3)
ISR-11	הצהרת בריאות תקופתית	החלטה איזה כלל חל
LIV-25	שיתוף כמה רכבים בקישור אחד	Traccar משתף רכב אחד לקישור; צריך שירות בשרת
ADM-22	עדכון חי (Realtime) בפורטל הנהג	שירות ה-Realtime של Supabase בשרת
REP-22	שעות בדשבורד לפי אזור הזמן של המשתמש	שינוי רוחבי; כל המשתמשים היום בישראל
ARC-06, ARC-08, ADM-15	מעבר מסכים ל-TanStack Query, חיפוש בשרת וטבלאות וירטואליות	ממשיכים כשנוגעים בכל מסך
INT-02, INT-03	מרכז אינטגרציות, אפליקציה בחנויות	החלטות (שלב 10)

הפירוט המלא של כל 213 הפריטים: `docs/roadmap/ITEMS.md`. סדר העבודה: `docs/roadmap/PROGRESS.md`.